DEVOPS DOCUMENTATION

Fullstack Deployment using Docker, Kubernetes, Terraform & Jenkins

Production-grade infrastructure on AWS — EKS, ALB, RDS, Redis, ECR, Route53 — deployed and managed entirely through code.

Deployed at: https://www.ankit.services AWS Account: 668076964228 Region: us-east-1

Todo Fullstack — DevOps Documentation

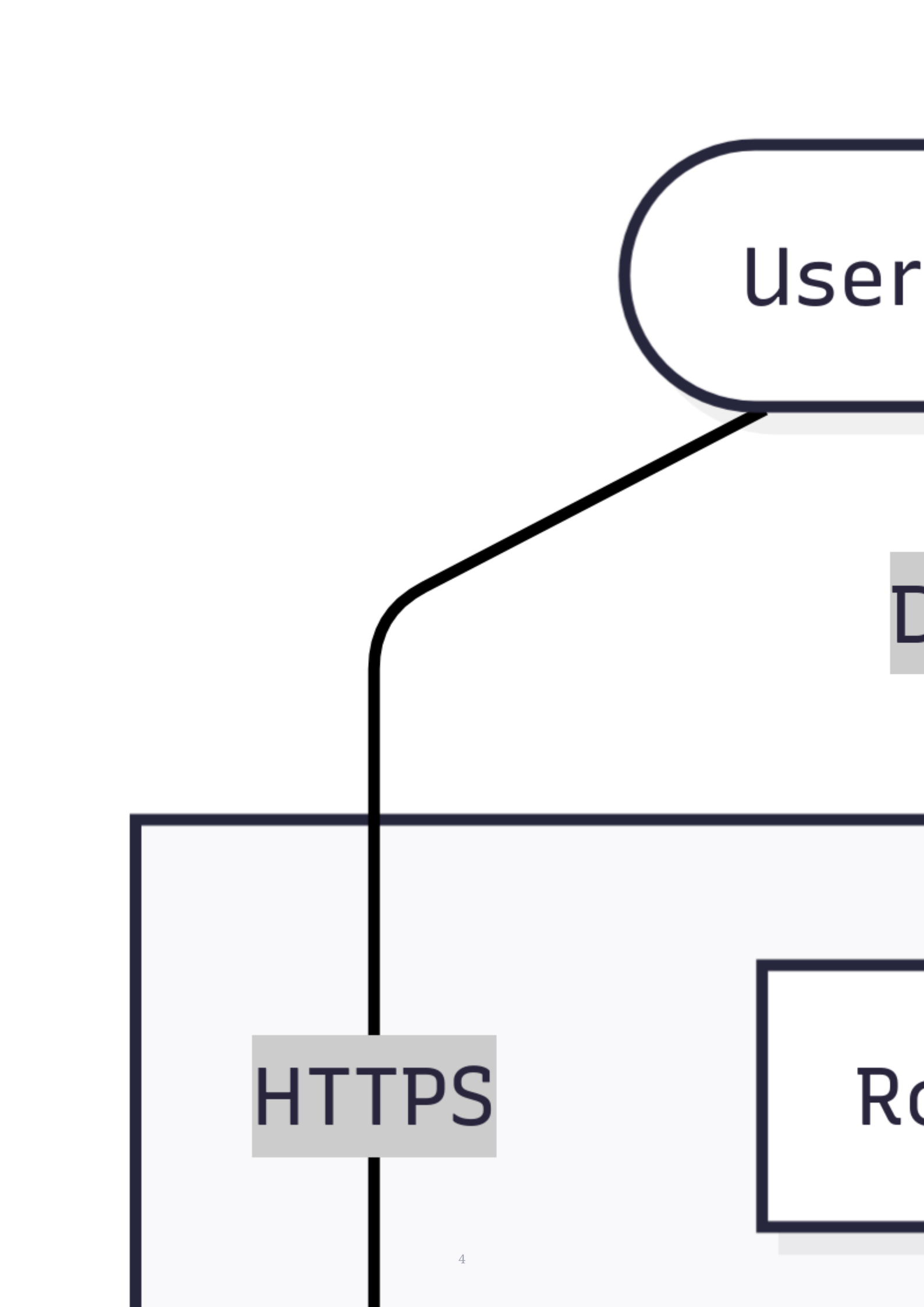
Overview

This project deploys a full-stack Todo application on AWS using a production-grade DevOps setup. The infrastructure is fully automated — from networking to container orchestration — using industry-standard tools.

Application: A Todo web app with a JavaScript frontend, Node.js backend, MySQL database, and Redis cache.

Deployed at: `https://www.ankit.services`

Architecture


User
HTTPS

Traffic Flow:

1. User visits `https://www.ankit.services`
2. Route53 resolves DNS → ALB IP
3. ALB terminates TLS, routes `/api/*` → Backend pods, `/` → Frontend pods
4. Backend reads/writes to RDS (MySQL) and Redis (cache/sessions)
5. Frontend is served as static files by the `serve` package

Tech Stack

Layer	Technology	Purpose
Infrastructure as Code	Terraform	Provision all AWS resources
Container Runtime	Docker	Package apps into images
Container Orchestration	Kubernetes (EKS)	Run and scale containers
CI/CD	Jenkins	Automate build, push, deploy
DNS	Route53	Domain management
Load Balancer	AWS ALB	HTTPS termination, path routing
Container Registry	ECR	Store Docker images
Database	RDS MySQL 8.0	Persistent data storage
Cache	ElastiCache Redis	Session cache
Object Storage	S3	Static assets
Monitoring	CloudWatch	Logs and metrics

Documentation Index

Document	Contents
Terraform	Infrastructure modules, variables, apply procedure
Docker	Dockerfiles, build strategy, ECR push
Kubernetes	Manifests, traffic flow, scaling, secrets
Jenkins	CI/CD pipeline, path-based triggers
Troubleshooting	Problems encountered and their solutions

Repository Structure

```
todo-fullstack/
├── app/
│   ├── backend/              # Node.js API source code
│   │   └── src/
│   │       ├── app.js        # Express server entry point
│   │       ├── db.js         # MySQL connection pool
│   │       ├── redis.js      # Redis client
│   │       ├── routes/       # API route definitions
│   │       └── controllers/  # Business logic
│   ├── frontend/             # Static HTML/JS/CSS
│   ├── docker/               # Dockerfiles and docker-compose
│   │   ├── Dockerfile.backend
│   │   ├── Dockerfile.frontend
│   │   └── docker-compose.yaml
│   └── k8s/                  # Kubernetes manifests
│       ├── namespace.yaml
│       ├── configmap.yaml
│       ├── secret.yaml
│       ├── deployment.yaml
│       ├── service.yaml
│       ├── ingress.yaml
│       ├── hpa.yaml
│       └── pdb.yaml
├── infra/
│   ├── ci-cd/
│   │   └── Jenkinsfile       # Pipeline definition
│   └── terraform/
│       ├── main.tf           # Root module — wires everything together
│       ├── variables.tf      # Input variables
│       ├── outputs.tf        # Exported values
│       ├── providers.tf      # AWS + TLS provider config
│       ├── versions.tf       # Provider version constraints
│       ├── environments/
│       │   ├── dev/          # Dev-specific variable values
│       │   └── prod/         # Prod-specific variable values
│       └── modules/
│           ├── vpc/          # Networking
│           ├── security-groups/
│           ├── eks/          # Kubernetes cluster
│           ├── ecr/          # Container registry
```

```
│           ├── rds/          # MySQL database
│           ├── redis/        # Cache
│           ├── alb/          # Load balancer
│           ├── asg/          # Auto Scaling Group policy
│           ├── autoscalling/ # Cluster Autoscaler IAM
│           ├── route53/      # DNS
│           ├── s3/           # Object storage
│           └── cloudwatch/   # Logging
└── docs/                     # This documentation
```

Quick Reference Commands

```
# Deploy infrastructure
cd infra/terraform
terraform apply -var-file=environments/dev/terraform.tfvars

# Configure kubectl
aws eks update-kubeconfig --region us-east-1 --name todo-tf-cluster-dev

# Deploy application
kubectl apply -f app/k8s/

# Check status
kubectl get pods -n todo
kubectl get ingress -n todo

# View logs
kubectl logs -n todo deployment/todo-backend
kubectl logs -n todo deployment/todo-frontend
```

Terraform — Infrastructure as Code

What is Terraform?

Terraform is a tool that lets you define your entire cloud infrastructure in code (`.tf` files). Instead of clicking through the AWS console to create resources, you describe what you want, and Terraform creates, updates, or deletes resources to match that description.

Why we use it:
- Every infrastructure resource is version-controlled in Git
- Reproducible — run the same code, get the same infrastructure every time
- Changes are reviewed before they are applied (`terraform plan`)
- Supports multiple environments (dev, prod) from the same codebase

---

How Terraform Works

```
Write .tf files  →  terraform plan  →  terraform apply  →  AWS Resources Created
     (code)          (preview)           (execute)
```

Terraform keeps track of what it has created in a **state file** (`terraform.tfstate`). On every apply, it compares the current state with the desired state and only changes what is different.

Project Structure

```
infra/terraform/
├── main.tf           ← Root: calls all modules
├── variables.tf      ← Declares all input variables
├── outputs.tf        ← Values exported after apply (endpoint URLs, IDs)
├── providers.tf      ← AWS provider configuration
├── versions.tf       ← Locks provider versions
├── environments/
│   ├── dev/terraform.tfvars   ← Dev values (small instance sizes, no multi-AZ)
│   └── prod/terraform.tfvars  ← Prod values (larger instances, multi-AZ, deletion protection)
└── modules/          ← Reusable building blocks
```

Each module is a folder with three standard files:
- `variables.tf` — inputs the module accepts
- `main.tf` — resources the module creates
- `outputs.tf` — values the module exposes to other modules

Modules

1. VPC (Virtual Private Cloud)

File: `modules/vpc/main.tf`

What it creates:
- 1 VPC (`10.0.0.0/16`) — an isolated network in AWS
- 2 public subnets — for the ALB (internet-facing)
- 2 private subnets — for EKS nodes, RDS, Redis (not reachable from internet)
- 1 Internet Gateway — allows public subnets to reach the internet
- 1 NAT Gateway — allows private subnet resources to reach the internet (for outbound, e.g., pulling Docker images)
- Route tables connecting subnets to the gateways

Why private subnets for nodes? Security. Application pods run on nodes that are not directly reachable from the internet. All traffic must go through the ALB.

```
Internet → Internet Gateway → Public Subnet (ALB)
                                      ↓
                                NAT Gateway
                                      ↓
                        Private Subnets (EKS nodes, RDS, Redis)
```

2. Security Groups

File: `modules/security-groups/main.tf`

Security groups are AWS firewalls at the resource level. Each resource only allows the traffic it needs.

Security Group	Inbound Allowed	Purpose
`eks-node-sg`	Port 3000 from ALB SG, self (node-to-node), ports 10250/443/1025-65535 from control plane	EKS worker nodes
`rds-sg`	Port 3306 from EKS node SG	MySQL — only pods can connect
`redis-sg`	Port 6379 from EKS node SG	Redis — only pods can connect
`alb-sg`	Port 80 and 443 from internet (0.0.0.0/0)	ALB — accepts public traffic

Key rule — `alb_to_nodes`: An `aws_security_group_rule` that allows the ALB to send traffic to port 3000 on worker nodes. Without this, the ALB health checks fail and pods are never marked healthy.

---

3. EKS (Elastic Kubernetes Service)

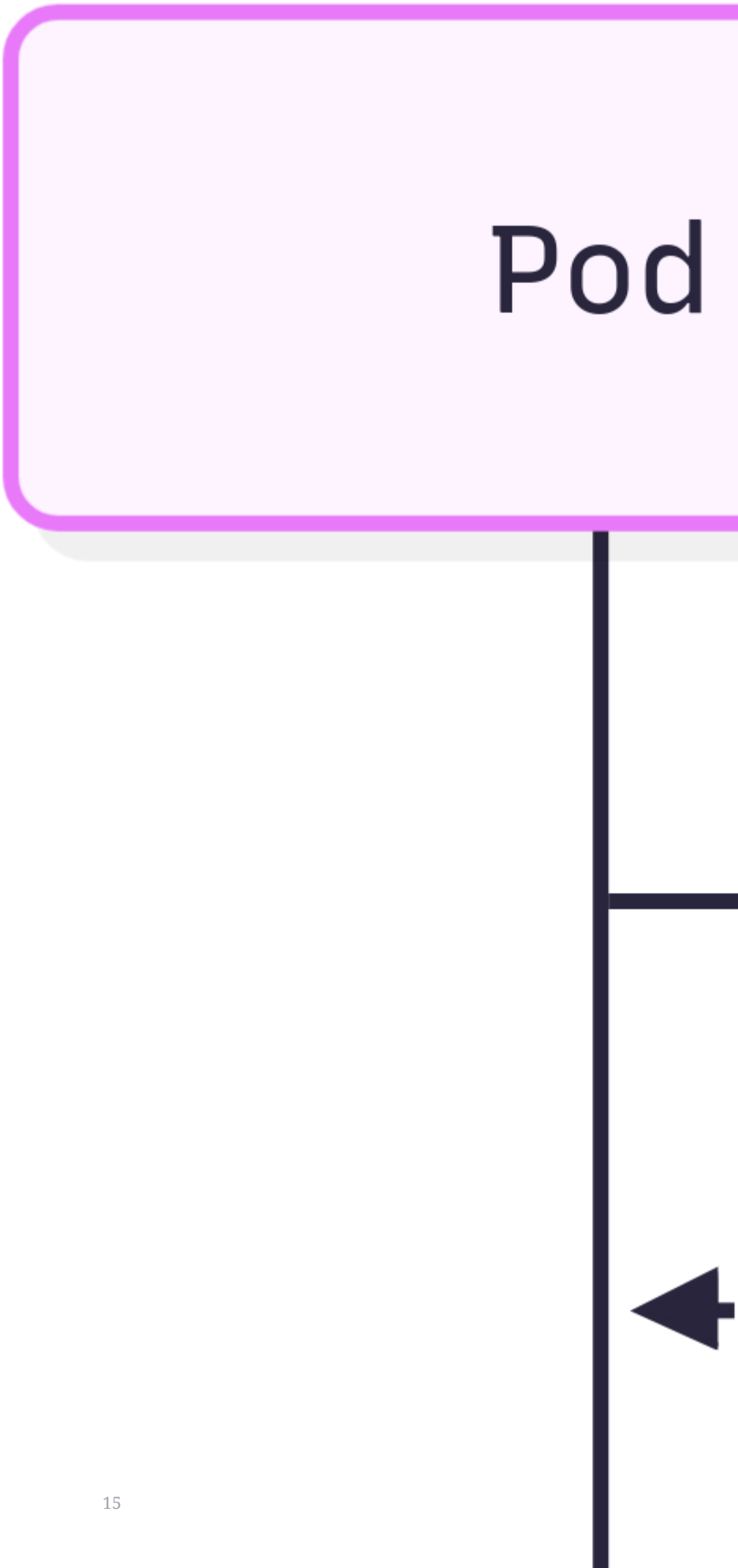
File: `modules/eks/main.tf`

What it creates:
- EKS cluster (the Kubernetes control plane — managed by AWS)
- OIDC provider (enables IRSA — pods assuming IAM roles)
- IAM role for the cluster
- IAM role for worker nodes
- Launch template (defines EC2 config for nodes: IMDSv2 only, encrypted gp3 storage, custom SG)
- Managed node group (the EC2 instances that run pods)
- Kubernetes addons: `vpc-cni`, `kube-proxy`, `coredns`, `aws-ebs-csi-driver`
- IRSA role for AWS Load Balancer Controller
- IRSA role for EBS CSI Driver
- Security group rules for control plane ↔ node communication

Why managed node group? AWS handles node provisioning, OS patching, and replacement on failure. We only define the instance type and scaling limits.

What is IRSA? IAM Roles for Service Accounts. Pods can assume IAM roles without storing credentials. The OIDC provider links a Kubernetes service account to an AWS IAM role using a JWT token.

```
Pod (with service account annotation)
    → Kubernetes issues OIDC token
    → Pod calls AWS STS with the token
    → STS validates token with OIDC provider
    → Returns temporary AWS credentials
    → Pod calls AWS APIs (e.g., ALB, S3)
```


Pod

Node group tags for Cluster Autoscaler:

```
"k8s.io/cluster-autoscaler/enabled"              = "true"
"k8s.io/cluster-autoscaler/todo-tf-cluster-dev"  = "owned"
```

These tags let the Cluster Autoscaler discover which ASG to scale.

---

4. ECR (Elastic Container Registry)

File: `modules/ecr/main.tf`

What it creates: One private Docker image repository per service.

Repository	Images stored
`todo-frontend`	Frontend static file server images
`todo-backend`	Node.js API server images

Why two repos? Separate repos allow independent pipelines. A backend change only triggers a backend image build. A shared repo would require tagging conventions (e.g., `frontend-v1`, `backend-v1`) which are error-prone.

Lifecycle policy: Automatically deletes old images, retaining only the latest N (configurable). Prevents storage costs from growing indefinitely.

---

5. RDS (Relational Database Service)

File: `modules/rds/main.tf`

What it creates: A managed MySQL 8.0 database instance.

Setting	Dev	Prod
Instance class	`db.t3.micro`	`db.t3.small`
Storage	20 GiB gp3	50 GiB gp3
Multi-AZ	No	Yes
Deletion protection	No	Yes
Final snapshot	Skipped	Taken

Why managed RDS? AWS handles: backups, point-in-time recovery, OS patches, minor version upgrades, and Multi-AZ failover. No DBA required.

Note: `performance_insights_enabled = false` — Performance Insights is not supported on `db.t3.micro`. It is supported on `db.t3.small` and above.

---

6. Redis (ElastiCache)

File: `modules/redis/main.tf`

What it creates: An ElastiCache replication group running Redis 7.1.

Used by the backend for session storage and caching. Reduces repeated database queries.

---

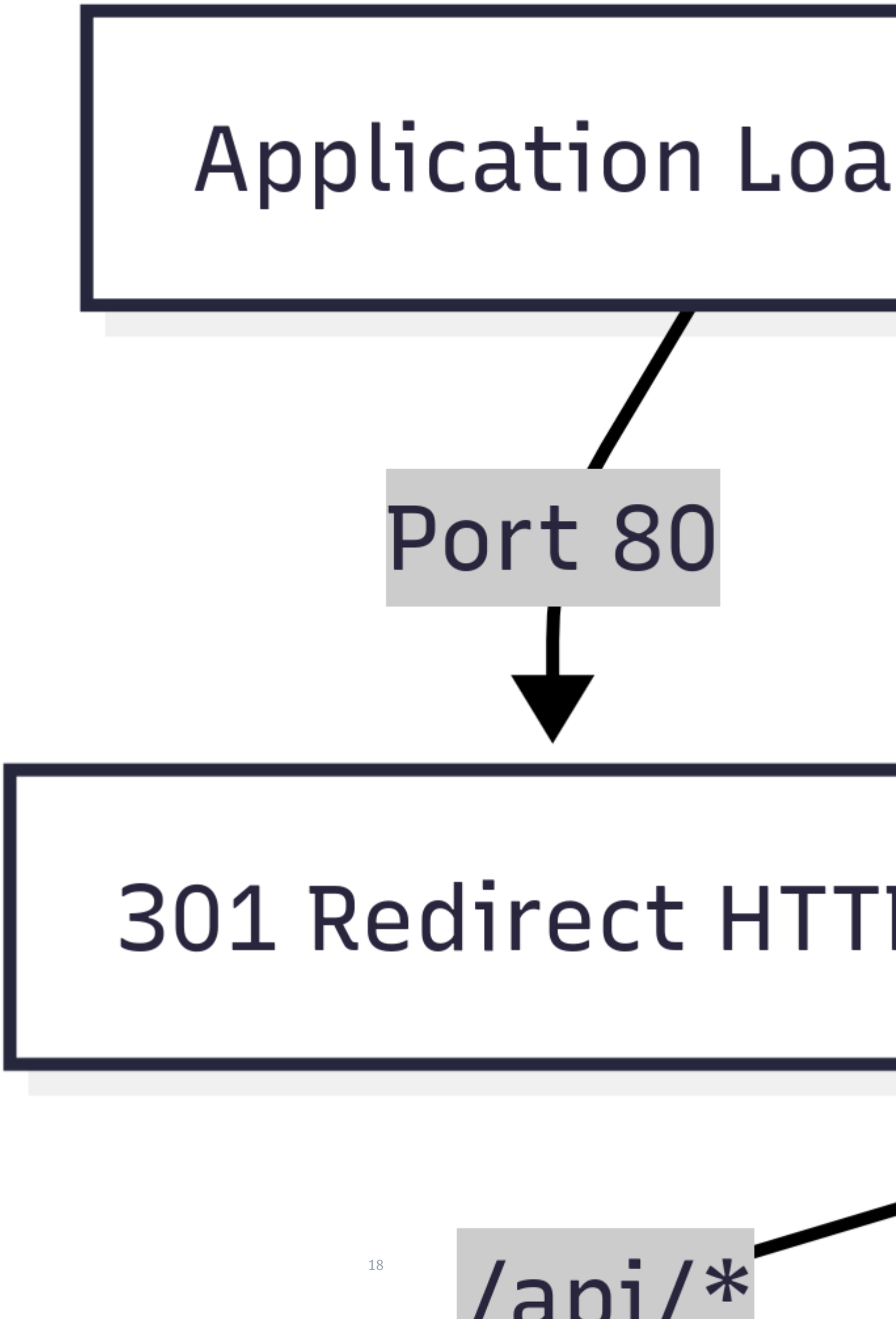
7. ALB (Application Load Balancer)

File: `modules/alb/main.tf`

What it creates:
- Internet-facing ALB in public subnets
- **Frontend target group** — `target_type = "ip"`, port 3000, health check path `/`
- **Backend target group** — `target_type = "ip"`, port 3000, health check path `/health`
- **HTTP listener (port 80)** — redirects all traffic to HTTPS (301)
- **HTTPS listener (port 443)** — forwards to frontend by default; path rule `/api/*` and `/health` forwards to backend

Why `target_type = "ip"`? The AWS Load Balancer Controller registers pod IPs directly into the target group (not EC2 instance IPs). This works because EKS uses the VPC CNI plugin which gives each pod a real VPC IP address.

```
ALB
├── Port 80  → redirect to 443
└── Port 443
    ├── /api/*   → Backend Target Group  → Backend pods (port 3000)
    ├── /health  → Backend Target Group  → Backend pods (port 3000)
    └── /        → Frontend Target Group → Frontend pods (port 3000)
```


Application Loa
Port 80
301 Redirect HTT
/api/*

8. ASG (Auto Scaling Group Policy)

File: `modules/asg/main.tf`

EKS automatically creates an ASG for the managed node group. This module finds that ASG and adds a CPU target tracking scaling policy to it.

What it does: If average CPU across all nodes exceeds 60%, AWS automatically adds more nodes. When CPU drops, nodes are removed (respecting `min_size`).

```
# Finds the EKS-created ASG by tags
data "aws_autoscaling_groups" "eks_nodes" {
  filter { name = "tag:eks:cluster-name" values = [var.cluster_name] }
  filter { name = "tag:eks:nodegroup-name" values = [var.node_group_name] }
}
```

---

9. Autoscalling (Cluster Autoscaler IRSA)

File: `modules/autoscalling/main.tf`

What it creates:
- IAM policy with permissions to describe and modify ASGs
- IRSA role that the Cluster Autoscaler pod assumes
- Policy attachment

What is the Cluster Autoscaler? A Kubernetes controller that watches for pods in `Pending` state (no nodes available) and increases the ASG desired count. It also removes underutilized nodes.

The Helm chart is installed separately after the cluster is up (Terraform cannot configure the Helm provider before the EKS cluster exists).

---

10. Route53

File: `modules/route53/main.tf`

What it creates: An A alias record pointing `www.ankit.services` to the ALB DNS name.

An alias record is an AWS-specific extension that works like a CNAME but can be used at the zone apex and resolves faster.

```
www.ankit.services  →  A alias  →  todo-alb-xxx.us-east-1.elb.amazonaws.com
```

11. S3

File: `modules/s3/main.tf`

A private S3 bucket for application assets. Configured with:
- Server-side encryption (AES-256)
- Versioning enabled
- All public access blocked
- Lifecycle rule to abort incomplete multipart uploads after 7 days

---

12. CloudWatch

File: `modules/cloudwatch/main.tf`

Pre-creates log groups so EKS starts shipping logs immediately:
- `/aws/eks/<cluster-name>/cluster` — control plane logs
- `/todo/dev/application` — application logs
- `/todo/dev/backend` — backend logs
- `/todo/dev/frontend` — frontend logs

Apply Procedure

Prerequisites

```
# Install tools
terraform --version   # ≥ 1.6.0
aws --version         # configured with appropriate IAM permissions

# Set sensitive variables
export TF_VAR_rds_password="YourStrongPassword123"
```

Commands

```
cd infra/terraform

# First time only — downloads providers and initializes modules
terraform init

# Preview changes before applying
terraform plan -var-file=environments/dev/terraform.tfvars

# Apply changes
terraform apply -var-file=environments/dev/terraform.tfvars

# View outputs (endpoints, ARNs)
terraform output

# Destroy all resources (use with caution)
terraform destroy -var-file=environments/dev/terraform.tfvars
```

State Management

Terraform stores state in `terraform.tfstate`. In production this must be stored remotely (S3 + DynamoDB lock) using `backend.tf`:

```
terraform {
  backend "s3" {
    bucket         = "your-tfstate-bucket"
    key            = "todo/dev/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"
  }
}
```

Importing Existing Resources

If a resource already exists in AWS but is not in Terraform state (e.g., from a failed/interrupted apply):

```
# Import an IAM role
terraform import -var-file=environments/dev/terraform.tfvars \
  module.eks.aws_iam_role.eks_cluster todo-dev-eks-cluster-role

# Import a CloudWatch log group
terraform import -var-file=environments/dev/terraform.tfvars \
  module.cloudwatch.aws_cloudwatch_log_group.app /todo/dev/application
```

Environment Differences

Variable	Dev	Prod
EKS node type	`t3.medium`	`t3.large`
Node desired	2	2
Node max	3	6
RDS class	`db.t3.micro`	`db.t3.small`
RDS Multi-AZ	false	true
RDS deletion protection	false	true
Redis nodes	1	1
Log retention	14 days	30 days

Docker — Containerisation

What is Docker?

Docker packages an application and all its dependencies (runtime, libraries, config) into a single portable unit called a **container image**. The image runs identically on any machine — a developer's laptop, a CI server, or a Kubernetes node in AWS.

Why we use it:
- "Works on my machine" is eliminated — the container is the machine
- Images are versioned and stored in ECR; any version can be rolled back to instantly
- EKS pulls images directly from ECR to run pods

Project Image Strategy

Image	Registry	Repository
Frontend	ECR	`668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-frontend`
Backend	ECR	`668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-backend`

Why two separate repositories?
- A backend code change should only trigger a backend image build — not rebuild the frontend
- Independent versioning: frontend can be on `v2.1`, backend on `v3.0`
- Avoids tag collision that occurs in a shared repo

Dockerfiles

Backend — `app/docker/Dockerfile.backend`

```
FROM node:20-alpine

# Run as non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app

# Copy package files first (layer cache — only re-runs npm ci when deps change)
COPY package*.json ./
RUN npm ci --omit=dev

# Copy source code
COPY src/ ./src/

# Fix file ownership
RUN chown -R appuser:appgroup /app

USER appuser

EXPOSE 3000
CMD ["node", "src/app.js"]
```

Key decisions:

Decision	Reason
`node:20-alpine`	Alpine Linux is ~5MB vs ~150MB for full Debian — smaller image, faster pulls
Non-root user	If a container is compromised, the attacker has no root privileges
`COPY package*.json` before `COPY src/`	Docker caches each layer. If source changes but `package.json` doesn't, npm install is not re-run — faster builds
`npm ci --omit=dev`	`ci` installs exactly what's in `package-lock.json` (reproducible). `--omit=dev` skips test frameworks, linters — smaller image

Build context: `app/backend/` — the Dockerfile expects `package.json` and `src/` at the root of the build context.

Frontend — `app/docker/Dockerfile.frontend`

```
FROM node:20-alpine

# Run as non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Install 'serve' — a lightweight static file server
RUN npm install -g serve@14

WORKDIR /app

# Copy all frontend files (index.html, app.js, style.css)
COPY . .

# Fix file ownership
RUN chown -R appuser:appgroup /app

USER appuser

EXPOSE 3000
CMD ["serve", "-s", ".", "-l", "3000"]
```

Key decisions:

Decision	Reason
`serve` package	The frontend is pure HTML/JS/CSS — no build step needed. `serve` hosts the files on port 3000
`COPY . .`	All frontend files are copied. Build context is `app/frontend/`
Port 3000	Both frontend and backend use port 3000. The ALB routes to separate target groups by URL path — not port

---

Build and Push Commands

Step 1 — Authenticate with ECR

ECR uses short-lived tokens (12 hours). Run this once per session:

```
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin \
    668076964228.dkr.ecr.us-east-1.amazonaws.com
```

Step 2 — Build Backend Image

```
docker build \
  -t 668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-backend:latest \
  -f app/docker/Dockerfile.backend \
  app/backend/
```

- `-f` specifies the Dockerfile location (it is not in the build context directory)
- Last argument `app/backend/` is the **build context** — the directory Docker sends to the daemon

Step 3 — Push Backend Image

```
docker push 668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-backend:latest
```

Step 4 — Build and Push Frontend Image

```
docker build \
  -t 668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-frontend:latest \
  -f app/docker/Dockerfile.frontend \
  app/frontend/

docker push 668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-frontend:latest
```

Local Development

For local testing without Kubernetes, use `docker-compose`:

```
docker compose -f app/docker/docker-compose.yaml up
```

This starts frontend, backend, MySQL, and Redis containers locally, wired together on a private Docker network. Useful for development before pushing to EKS.

Image Lifecycle in ECR

ECR retains the last **10 images** (dev: 5) per repository, controlled by the lifecycle policy Terraform creates:

```
{
  "rules": [{
    "rulePriority": 1,
    "description": "Keep last 10 tagged images",
    "selection": {
      "tagStatus": "tagged",
      "countType": "imageCountMoreThan",
      "countNumber": 10
    },
    "action": { "type": "expire" }
  }]
}
```

Older images are automatically deleted. This keeps storage costs low and prevents the registry from accumulating hundreds of stale images.

Tagging Strategy

Currently using `latest` tag for simplicity. In production, best practice is to tag with the Git commit SHA:

```
IMAGE_TAG=$(git rev-parse --short HEAD)
docker build -t .../todo-backend:$IMAGE_TAG ...
docker push .../todo-backend:$IMAGE_TAG

# Also push as latest for convenience
docker tag .../todo-backend:$IMAGE_TAG .../todo-backend:latest
docker push .../todo-backend:latest
```

This makes every build traceable back to an exact commit.

Kubernetes — Container Orchestration

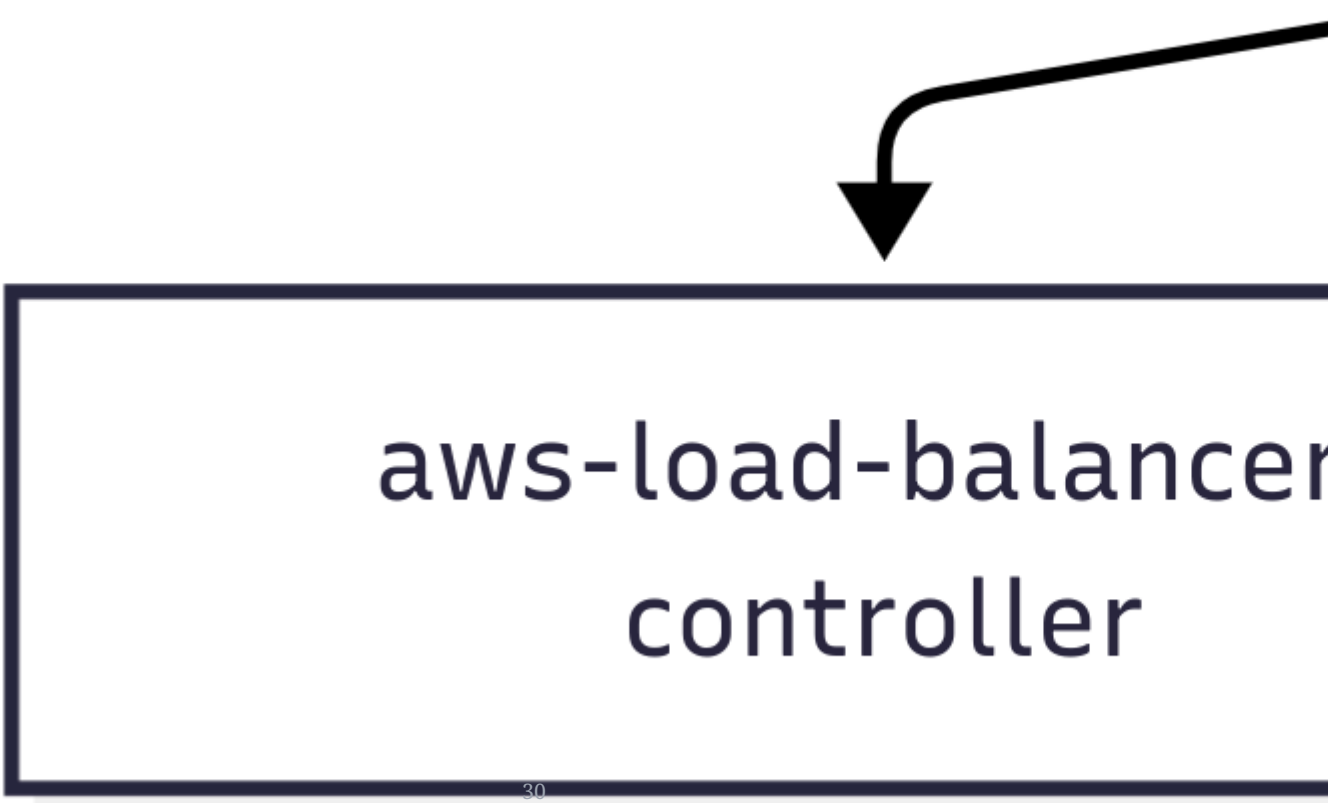
What is Kubernetes?

Kubernetes (K8s) is a system that manages containers at scale. Instead of manually running `docker run` on servers, you declare what you want (e.g., "run 2 copies of the backend container") and Kubernetes continuously ensures that state is maintained — restarting crashed containers, rescheduling on healthy nodes, and scaling up/down.

Why we use EKS (Elastic Kubernetes Service)?
AWS manages the Kubernetes control plane (the master nodes). We only manage the worker nodes where our pods run. This eliminates the operational burden of running etcd, the API server, and scheduler ourselves.

Cluster Overview

```
EKS Cluster (todo-tf-cluster-dev)
│
├── kube-system namespace
│   ├── aws-load-balancer-controller  ← watches Ingress, manages ALB target groups
│   ├── coredns                       ← DNS resolution inside the cluster
│   ├── kube-proxy                    ← network rules on each node
│   └── vpc-cni                       ← gives each pod a real VPC IP
│
└── todo namespace
    ├── todo-frontend (Deployment, 2 replicas)
    ├── todo-backend  (Deployment, 2 replicas)
    ├── todo-frontend-svc (Service, ClusterIP)
    ├── todo-backend-svc  (Service, ClusterIP)
    ├── todo-ingress (Ingress, ALB class)
    ├── todo-config  (ConfigMap)
    ├── todo-secret  (Secret)
    ├── HPA (HorizontalPodAutoscaler × 2)
    └── PDB (PodDisruptionBudget × 2)
```


aws-load-balancer
controller

Manifests Explained

1. Namespace — `namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: todo
```

What it does: Creates an isolated workspace called `todo`. All application resources live here.

Why: Namespaces prevent resource name collisions and allow per-namespace access control and resource quotas. The `kube-system` namespace is for cluster components — keeping our app in `todo` separates concerns clearly.

---

2. ConfigMap — `configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: todo-config
  namespace: todo
data:
  PORT: "3000"
  DB_NAME: "todo_db"
  ALLOWED_ORIGIN: "https://www.ankit.services"
```

What it does: Stores non-sensitive configuration as key-value pairs, injected into pods as environment variables.

Why separate from the image? Configuration changes (e.g., updating `ALLOWED_ORIGIN`) don't require rebuilding the Docker image — just update the ConfigMap and roll the deployment.

Rule: ConfigMap = safe to commit to Git. Secret = never commit plain text.

---

3. Secret — `secret.yaml`

```
apiVersion: v1
kind: Secret
metadata:
  name: todo-secret
  namespace: todo
type: Opaque
data:
  DB_HOST:     <base64 encoded>
  DB_PORT:     <base64 encoded>
  DB_USER:     <base64 encoded>
  DB_PASSWORD: <base64 encoded>
  REDIS_HOST:  <base64 encoded>
  REDIS_PORT:  <base64 encoded>
```

What it does: Stores sensitive values. Kubernetes stores them separately from regular config and only mounts them in pods that need them.

Important: Base64 is encoding, not encryption. The values can be decoded with `base64 -d`. For production, use AWS Secrets Manager with the External Secrets Operator to inject secrets at runtime.

How to generate values:

```
# RDS endpoint (remove the :3306 port suffix)
terraform output -raw rds_endpoint | cut -d: -f1 | base64

# Redis endpoint
terraform output -raw redis_primary_endpoint | base64

# Password
echo -n "YourPassword" | base64
```

---

4. Deployment — `deployment.yaml`

The most important manifest. Defines how pods are created and managed.

Frontend Deployment:

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: todo-app-frontend
  template:
    spec:
      containers:
      - name: frontend
        image: 668076964228.dkr.ecr.us-east-1.amazonaws.com/todo-frontend:latest
        ports:
        - containerPort: 3000
        envFrom:
        - configMapRef:
            name: todo-config
```

Backend Deployment: Same structure but also mounts the Secret:

```
        envFrom:
        - configMapRef:
            name: todo-config
        - secretRef:
            name: todo-secret
```

Key settings:

Setting	Value	Reason
`replicas: 2`	2 pods each	High availability — if one pod crashes, the other handles traffic
`envFrom configMapRef`	Injects all ConfigMap keys as env vars	Decouples config from image
`envFrom secretRef`	Injects all Secret keys as env vars	Keeps credentials out of the image

Liveness vs Readiness Probes (if configured):
- **Liveness:** If this fails, Kubernetes restarts the container
- **Readiness:** If this fails, Kubernetes stops sending traffic to this pod (but doesn't restart it)

---

5. Service — `service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: todo-frontend-svc
  namespace: todo
spec:
  type: ClusterIP
  selector:
    app: todo-app-frontend
  ports:
  - port: 80
    targetPort: 3000
```

What it does: Creates a stable internal IP and DNS name for a group of pods. `todo-frontend-svc.todo.svc.cluster.local` always resolves to one of the running frontend pods.

Why ClusterIP (not LoadBalancer)? The ALB is our external entry point, managed by the Ingress. We don't need a separate AWS load balancer per service. ClusterIP is internal-only — cheaper and simpler.

Port mapping: The Service listens on port 80 internally but forwards to port 3000 on the pods (where the app runs).

---

6. Ingress — `ingress.yaml`

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: todo-ingress
  namespace: todo
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/load-balancer-arn: arn:aws:elasticloadbalancing:...
spec:
  ingressClassName: alb
  rules:
  - host: www.ankit.services
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: todo-backend-svc
            port:
              number: 80
      - path: /health
        pathType: Prefix
        backend:
          service:
            name: todo-backend-svc
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: todo-frontend-svc
            port:
              number: 80
```

What it does: The AWS Load Balancer Controller reads this resource and configures the ALB's listener rules accordingly.

Key annotations:

Annotation	Value	Purpose
`scheme: internet-facing`	—	ALB is reachable from the internet
`target-type: ip`	—	ALB registers pod IPs directly (not EC2 IPs)
`certificate-arn`	ACM ARN	TLS certificate for HTTPS
`ssl-redirect: 443`	—	HTTP requests are redirected to HTTPS
`load-balancer-arn`	existing ALB ARN	Tells controller to use this existing ALB instead of creating a new one

Path routing:

```
https://www.ankit.services/api/todos  → todo-backend-svc → backend pods
https://www.ankit.services/health     → todo-backend-svc → backend pods
https://www.ankit.services/           → todo-frontend-svc → frontend pods
```

---

7. HPA (HorizontalPodAutoscaler) — `hpa.yaml`

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: todo-backend-hpa
  namespace: todo
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: todo-backend
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

What it does: Automatically increases/decreases the number of backend pods based on CPU usage.

How it works:

```
CPU > 70% for sustained period → HPA adds pods (up to maxReplicas: 6)
CPU < 70% and stable          → HPA removes pods (down to minReplicas: 2)
```

Two levels of autoscaling:
1. **HPA** (this) — scales pods within a node
2. **Cluster Autoscaler** — adds/removes nodes when pods can't be scheduled

---

8. PDB (PodDisruptionBudget) — `pdb.yaml`

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: todo-backend-pdb
  namespace: todo
spec:
  minAvailable: 1
  selector:
    matchLabels:
      app: todo-app-backend
```

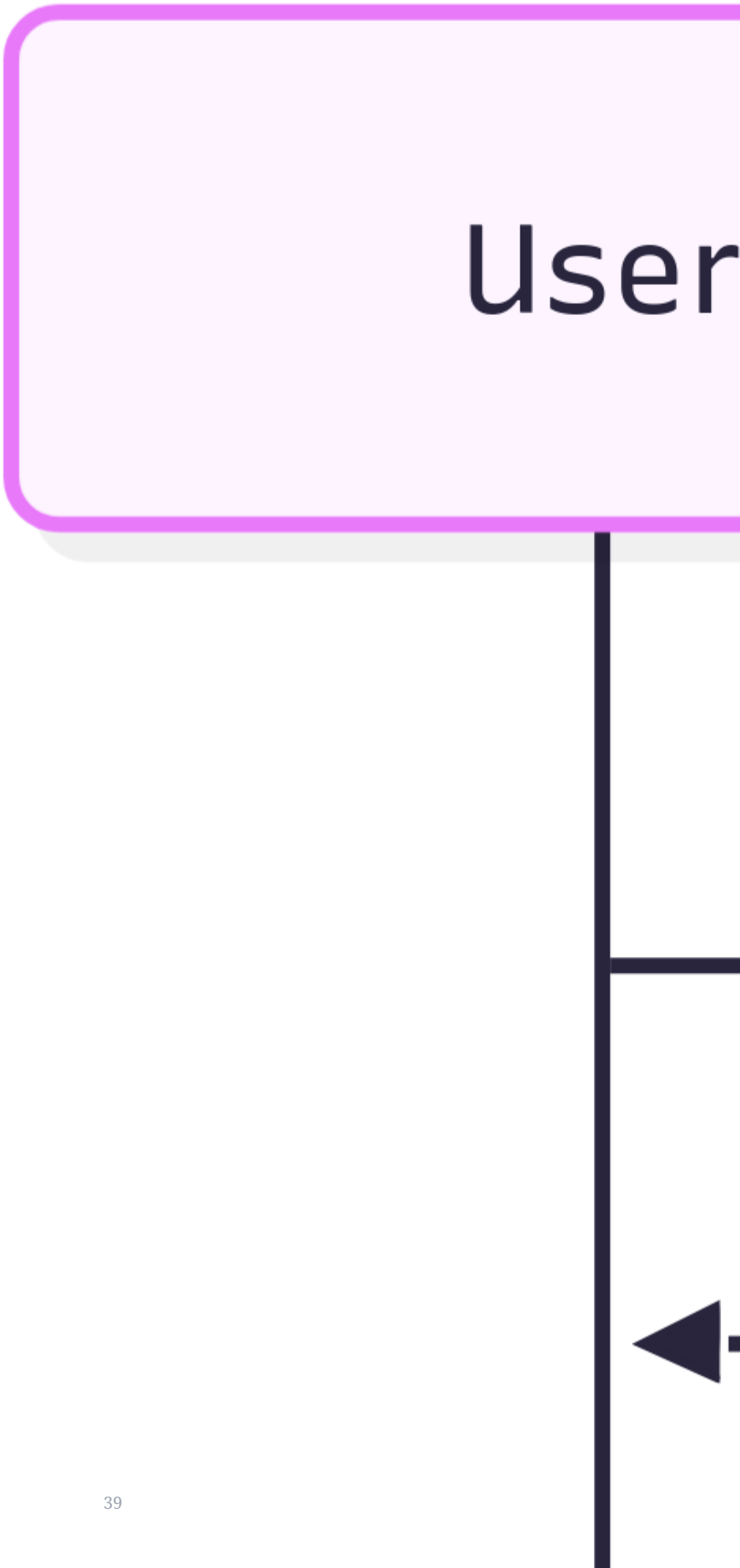
What it does: Guarantees that at least 1 backend pod is always running during voluntary disruptions (node drains, upgrades).

Why it matters: Without a PDB, Kubernetes could drain a node and temporarily leave zero backend pods running. With `minAvailable: 1`, Kubernetes waits until a replacement pod is running before draining the next one.

Traffic Flow — Complete Picture

```
User request: GET https://www.ankit.services/api/todos


1. DNS:     Route53 → ALB IP address
2. TLS:     ALB terminates HTTPS using ACM certificate
3. Routing: ALB rule matches /api/* → Backend Target Group
4. Target:  ALB selects a healthy backend pod IP (e.g., 10.0.11.76:3000)
5. Network: Traffic flows directly to pod via VPC CNI
6. App:     backend/src/app.js handles the request
7. DB:      Queries RDS MySQL at todo-db-dev.xxx.rds.amazonaws.com:3306
8. Cache:   Reads/writes Redis at todo-redis-dev.xxx.cache.amazonaws.com:6379
9. Response: JSON returned → ALB → User
```


User

Useful Commands

```
# Set namespace shortcut
alias k="kubectl -n todo"

# Check everything
kubectl get all -n todo

# Pod logs
kubectl logs -n todo deployment/todo-backend --tail=50 -f
kubectl logs -n todo deployment/todo-frontend --tail=50

# Describe a failing pod
kubectl describe pod -n todo <pod-name>

# Shell into a pod (debugging)
kubectl exec -it -n todo deployment/todo-backend -- sh

# Check HPA status
kubectl get hpa -n todo

# Check ingress address
kubectl get ingress -n todo

# Force restart a deployment (picks up new image or config)
kubectl rollout restart deployment/todo-backend -n todo
kubectl rollout status deployment/todo-backend -n todo

# Check events (useful for diagnosing issues)
kubectl get events -n todo --sort-by='.lastTimestamp'
```

Updating the Application

New image (code change):

```
# Build and push new image
docker build -t .../todo-backend:latest -f app/docker/Dockerfile.backend app/backend/
docker push .../todo-backend:latest

# Restart the deployment to pull the new image
kubectl rollout restart deployment/todo-backend -n todo
```

Config change (no rebuild needed):

```
# Edit configmap.yaml, then:
kubectl apply -f app/k8s/configmap.yaml
kubectl rollout restart deployment/todo-backend -n todo
```

Secret change:

```
# Edit secret.yaml with new base64 values, then:
kubectl apply -f app/k8s/secret.yaml
kubectl rollout restart deployment/todo-backend -n todo
```

Jenkins — CI/CD Pipeline

What is Jenkins?

Jenkins is an open-source automation server. When code is pushed to Git, Jenkins automatically builds Docker images, pushes them to ECR, and rolls out the new version to EKS — all without manual steps.

Why Jenkins (not GitHub Actions)?
- Self-hosted: runs inside your own infrastructure, no per-minute billing
- Works with any Git server (GitHub, GitLab, Bitbucket, self-hosted)
- Mature plugin ecosystem — AWS credentials, Kubernetes, Docker all have first-class support
- Pipeline-as-code: the Jenkinsfile is version-controlled alongside the application

---

Pipeline File Location

```
infra/ci-cd/Jenkinsfile
```

The Jenkinsfile is the single source of truth for the CI/CD process. Jenkins reads it from the repo on every run.

Environment Variables

Defined once at the top, used throughout all stages:

```
environment {
    AWS_REGION   = 'us-east-1'
    ECR_REGISTRY = '668076964228.dkr.ecr.us-east-1.amazonaws.com'
    ECR_FRONTEND = "${ECR_REGISTRY}/todo-frontend"
    ECR_BACKEND  = "${ECR_REGISTRY}/todo-backend"
    EKS_CLUSTER  = 'todo-tf-cluster'
    IMAGE_TAG    = "v${BUILD_NUMBER}"    // e.g. v42, v43, v44
    KUBE_NS      = 'todo'
    TF_DIR       = 'infra/terraform'
}
```

`IMAGE_TAG = "v${BUILD_NUMBER}"` — Jenkins auto-increments `BUILD_NUMBER` on every run. This means every build produces a unique, traceable image tag. The image is also tagged `latest` for convenience.

Pipeline Structure

```
Checkout  (+ DinD daemon readiness check)
    |
    ├──────────────────────────────────────────┐
    |                                          |
  Frontend (parallel)                      Backend (parallel)
    ├── Build Frontend  (if app/frontend/** changed)   ├── Build Backend  (if app/backend/** changed)
    ├── Push Frontend   (if app/frontend/** changed)   ├── Push Backend   (if app/backend/** changed)
    └── Deploy Frontend (if app/frontend/** changed)   └── Deploy Backend (if app/backend/** changed)
    |                                          |
    └──────────────────────────────────────────┘
    |
Terraform Init     (if infra/** changed)
Terraform Validate (if infra/** changed)
Terraform Plan     (if infra/** changed)
Terraform Apply    (if infra/** changed)
    |
Apply K8s Manifests  (if infra/** OR app/k8s/** changed)
    |
Cleanup
```

Stage-by-Stage Breakdown

Stage 1 — Checkout

```
stage('Checkout') {
    steps {
        checkout scm
        // Wait for DinD daemon to be ready before any Docker commands
        retry(15) {
            sleep(time: 3, unit: 'SECONDS')
            sh 'docker info > /dev/null 2>&1'
        }
    }
}
```

Clones the repository at the commit that triggered the pipeline. `scm` refers to the source control configuration set in the Jenkins job.

After checkout, the pipeline waits for the Docker-in-Docker sidecar daemon to be ready before any stage tries to run `docker build`. The agent pod's DinD container can take a few seconds to initialise after the pod starts. Without this check, the first `docker build` call would fail with `"Cannot connect to the Docker daemon"`. The `retry(15)` block polls every 3 seconds for up to 45 seconds — enough time for DinD to come up on any reasonably-sized node.

Stage 2 — App (Parallel)

The frontend and backend pipelines run simultaneously. This halves the total build time when both change.

Changeset Guards (`when { changeset '...' }`)

Every sub-stage is wrapped in a `when` condition:

```
when { changeset 'app/frontend/**' }
```

Why this matters: If you push a change to only the backend (`app/backend/`), Jenkins skips all three frontend stages entirely — no unnecessary image rebuild, no unnecessary deployment.

Push changes to...	Frontend stages	Backend stages	Terraform stages	Apply K8s Manifests
`app/frontend/`	Run	Skip	Skip	Skip
`app/backend/`	Skip	Run	Skip	Skip
`infra/`	Skip	Skip	Run	Run
`app/k8s/`	Skip	Skip	Skip	Run
`app/frontend/` + `app/backend/`	Run	Run	Skip	Skip
`infra/` + `app/k8s/`	Skip	Skip	Run	Run

Build Frontend

```
sh """
    docker build \
        -f app/docker/Dockerfile.frontend \
        -t ${ECR_FRONTEND}:${IMAGE_TAG} \
        app/frontend/
"""
```

- `-f app/docker/Dockerfile.frontend` — Dockerfile is not in the build context directory; `-f` points to it explicitly
- `-t ...frontend:v42` — tags with the build number
- `app/frontend/` — build context (everything Docker can access during build)

Push Frontend

```
withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
                  credentialsId: 'aws-creds']]) {
    sh """
        aws ecr get-login-password --region ${AWS_REGION} | \
            docker login --username AWS --password-stdin ${ECR_REGISTRY}
        docker push ${ECR_FRONTEND}:${IMAGE_TAG}
        docker tag  ${ECR_FRONTEND}:${IMAGE_TAG} ${ECR_FRONTEND}:latest
        docker push ${ECR_FRONTEND}:latest
    """
}
```

ECR tokens expire after 12 hours, so the pipeline logs in fresh every run. The image is pushed twice: once with the versioned tag (`v42`) and once as `latest`. The entire block is wrapped in `withCredentials` so the `aws ecr get-login-password` call has valid IAM credentials — the same `aws-creds` credential used by the Terraform stages.

Deploy Frontend

```
withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
                  credentialsId: 'aws-creds']]) {
    sh """
        aws eks update-kubeconfig \
            --region ${AWS_REGION} --name ${EKS_CLUSTER}
        kubectl set image deployment/todo-frontend \
            todo-frontend=${ECR_FRONTEND}:${IMAGE_TAG} \
            -n ${KUBE_NS}
        kubectl rollout status deployment/todo-frontend \
            -n ${KUBE_NS} --timeout=180s \
        || (kubectl rollout undo deployment/todo-frontend -n ${KUBE_NS} && exit 1)
    """
}
```

How the rollout works:

1. `aws eks update-kubeconfig` — configures `kubectl` to talk to the EKS cluster using IAM credentials
2. `kubectl set image` — patches the deployment to use the new image tag; Kubernetes starts a rolling update
3. `kubectl rollout status --timeout=180s` — waits up to 3 minutes for the rollout to complete
4. `|| (kubectl rollout undo ... && exit 1)` — if the rollout fails or times out, Jenkins immediately rolls back to the previous working image and marks the build failed

This is an **automatic rollback** — no manual intervention needed if the new image crashes or fails health checks.

Backend stages are identical — same build, push, deploy pattern with `app/backend/**` changeset guard and `todo-backend` deployment name.

Stages 3–6 — Terraform (Infrastructure Changes)

These stages only run when files under `infra/` change.

Terraform Init

```
withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
                  credentialsId: 'aws-creds']]) {
    sh "terraform -chdir=${TF_DIR} init -input=false"
}
```

Downloads providers and modules. `withCredentials` injects `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` from the Jenkins credential store — credentials are never hardcoded.

Terraform Validate

```
sh "terraform -chdir=${TF_DIR} validate"
```

Validates HCL syntax and internal consistency without connecting to AWS. Fast — runs in seconds.

Terraform Plan

```
sh """
    terraform -chdir=${TF_DIR} plan \
        -var-file=environments/dev/terraform.tfvars \
        -out=tfplan -input=false
"""
```

Computes the diff between current state and desired state. Saves the plan to `tfplan` so Apply executes exactly what was planned.

Terraform Apply

```
sh "terraform -chdir=${TF_DIR} apply -input=false tfplan"
```

Executes the saved plan. Because the plan is pre-approved, this runs non-interactively.

Note: In production pipelines, you would add a manual approval gate between Plan and Apply:

```
stage('Approve Terraform Apply') {
    steps {
        input message: 'Review the Terraform plan. Proceed with apply?'
    }
}
```

Stage 7 — Apply K8s Manifests

```
stage('Apply K8s Manifests') {
    when {
        anyOf {
            changeset 'infra/**'
            changeset 'app/k8s/**'
        }
    }
    steps {
        withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
                          credentialsId: 'aws-creds']]) {
            sh """
                aws eks update-kubeconfig --region ${AWS_REGION} --name ${EKS_CLUSTER}

                FRONTEND_TG_ARN=\$(terraform -chdir=${TF_DIR} output -raw frontend_tg_arn)
                BACKEND_TG_ARN=\$(terraform -chdir=${TF_DIR} output -raw backend_tg_arn)
                BACKEND_IRSA_ROLE_ARN=\$(terraform -chdir=${TF_DIR} output -raw todo_backend_irsa_role_arn)
                SECRET_NAME=\$(terraform -chdir=${TF_DIR} output -raw secrets_manager_secret_name)

                sed -e "s|__FRONTEND_TG_ARN__|\${FRONTEND_TG_ARN}|g" \
                    -e "s|__BACKEND_TG_ARN__|\${BACKEND_TG_ARN}|g" \
                    app/k8s/targetgroupbinding.yaml | kubectl apply -f -

                sed "s|__BACKEND_IRSA_ROLE_ARN__|\${BACKEND_IRSA_ROLE_ARN}|g" \
                    app/k8s/serviceaccount.yaml | kubectl apply -f -

                sed "s|__SECRET_NAME__|\${SECRET_NAME}|g" \
                    app/k8s/secretproviderclass.yaml | kubectl apply -f -
            """
        }
    }
}
```

This stage solves the problem of Kubernetes manifests that reference AWS resource ARNs which are only known after Terraform runs. Rather than manually editing manifest files after every `terraform apply`, the manifests store placeholder tokens and the pipeline injects the real values at deploy time using `sed`.

When it runs: any change under `infra/` (Terraform resources changed, so ARNs may have changed) or `app/k8s/` (manifest templates were edited).

What it does — step by step:

1. Configures `kubectl` to reach the EKS cluster
2. Reads four values from Terraform output at pipeline time:

Terraform output	Used in manifest
`frontend_tg_arn`	`targetgroupbinding.yaml` — ALB target group for frontend pods
`backend_tg_arn`	`targetgroupbinding.yaml` — ALB target group for backend pods
`todo_backend_irsa_role_arn`	`serviceaccount.yaml` — IAM role the backend pod assumes
`secrets_manager_secret_name`	`secretproviderclass.yaml` — which Secrets Manager secret to mount

1. Applies each manifest after substituting the placeholder tokens:

Manifest	Placeholder(s) replaced
`targetgroupbinding.yaml`	`__FRONTEND_TG_ARN__`, `__BACKEND_TG_ARN__`
`serviceaccount.yaml`	`__BACKEND_IRSA_ROLE_ARN__`
`secretproviderclass.yaml`	`__SECRET_NAME__`

Why this approach matters: These three manifests contain AWS ARNs that are account- and environment-specific. Hardcoding them means the repo can't be cloned and deployed to a fresh account without manual edits. Using `__PLACEHOLDER__` tokens keeps the manifests portable — Terraform always knows the correct values, and the pipeline bridges the two systems automatically. No one needs to copy-paste ARNs by hand after an infrastructure change.

Stage 8 — Cleanup

```
sh """
    docker rmi ${ECR_FRONTEND}:${IMAGE_TAG} ${ECR_FRONTEND}:latest || true
    docker rmi ${ECR_BACKEND}:${IMAGE_TAG}  ${ECR_BACKEND}:latest  || true
    rm -f ${TF_DIR}/tfplan || true
"""
```

Removes locally built images from the Jenkins agent and deletes the saved Terraform plan. The `|| true` prevents the stage from failing if an image was never built (e.g., frontend-only change means backend image doesn't exist).

Post Section

```
post {
    success { echo "Pipeline ${IMAGE_TAG} completed successfully." }
    failure { echo 'Pipeline failed — check stage logs above.' }
}
```

Runs after all stages complete regardless of outcome. In production, replace `echo` with Slack or email notifications:

```
post {
    failure {
        slackSend channel: '#alerts', message: "Build ${IMAGE_TAG} FAILED: ${env.BUILD_URL}"
    }
}
```

Jenkins Setup Requirements

1. Jenkins Credentials

The pipeline requires two credentials stored in Jenkins (Manage Jenkins → Credentials → Global):

ID	Type	Used in	Contents
`aws-creds`	AWS Credentials	Push, Deploy, and all Terraform stages	IAM access key + secret key
`rds-password`	Secret text	`environment { TF_VAR_rds_password }`	RDS master password, passed to Terraform as a variable

The `aws-creds` IAM user/role needs permissions for: ECR (push/pull), EKS (update-kubeconfig, kubectl), S3 (Terraform state), and the full set of Terraform-managed resources.

The `rds-password` credential is injected as `TF_VAR_rds_password` at the top of the pipeline so Terraform can pass it to the RDS module without the password ever appearing in `.tfvars` files or pipeline logs.

2. Required Plugins

Plugin	Purpose
Pipeline	Jenkinsfile support
Git	Source checkout
AWS Credentials	`AmazonWebServicesCredentialsBinding`
Docker Pipeline	`docker build/push` in pipeline steps

3. Jenkins Agent Requirements

The agent (the machine that runs build steps) must have these tools installed:
- `docker` (with daemon running)
- `aws` CLI (v2)
- `kubectl`
- `terraform` (>= 1.6.0)

End-to-End Flow — Code Change to Production

```
Developer pushes to main branch
    |
    ▼
GitHub/GitLab notifies Jenkins (webhook)
    |
    ▼
Jenkins reads Jenkinsfile
    |
    ▼
Checkout stage pulls latest commit + waits for DinD daemon ready
    |
    ▼
changeset 'app/backend/**' matches? YES
    |
    ▼
Build Backend:  docker build → image tagged v43
Push Backend:   (withCredentials: aws-creds) ECR login → docker push v43 + latest
Deploy Backend: (withCredentials: aws-creds) kubectl set image → rolling update starts
                kubectl rollout status waits 180s
                    ├── SUCCESS → pipeline continues
                    └── FAIL    → kubectl rollout undo (v42 restored) + pipeline fails
    |
    ▼
changeset 'infra/**' matches? NO → Apply K8s Manifests skipped
    |
    ▼
Cleanup: docker rmi local images
    |
    ▼
post { success } or post { failure }
```

Infrastructure + manifest change (e.g., after `terraform apply`):

```
changeset 'infra/**' matches? YES
    |
    ▼
Terraform Init → Validate → Plan → Apply  (all withCredentials: aws-creds)
    |
    ▼
Apply K8s Manifests:
    Read terraform output → frontend_tg_arn, backend_tg_arn,
                            todo_backend_irsa_role_arn, secrets_manager_secret_name
    sed-inject ARNs into targetgroupbinding.yaml → kubectl apply
    sed-inject role ARN into serviceaccount.yaml  → kubectl apply
    sed-inject secret name into secretproviderclass.yaml → kubectl apply
    |
    ▼
Cleanup
```


De


g


Git


W

Total time for a backend-only change: ~3-5 minutes (build + push + rolling rollout)

Image Tagging Strategy

```
v43 (build number)  ←  traceability: maps to a specific Jenkins build and Git commit
latest              ←  convenience: always points to the most recently deployed image
```

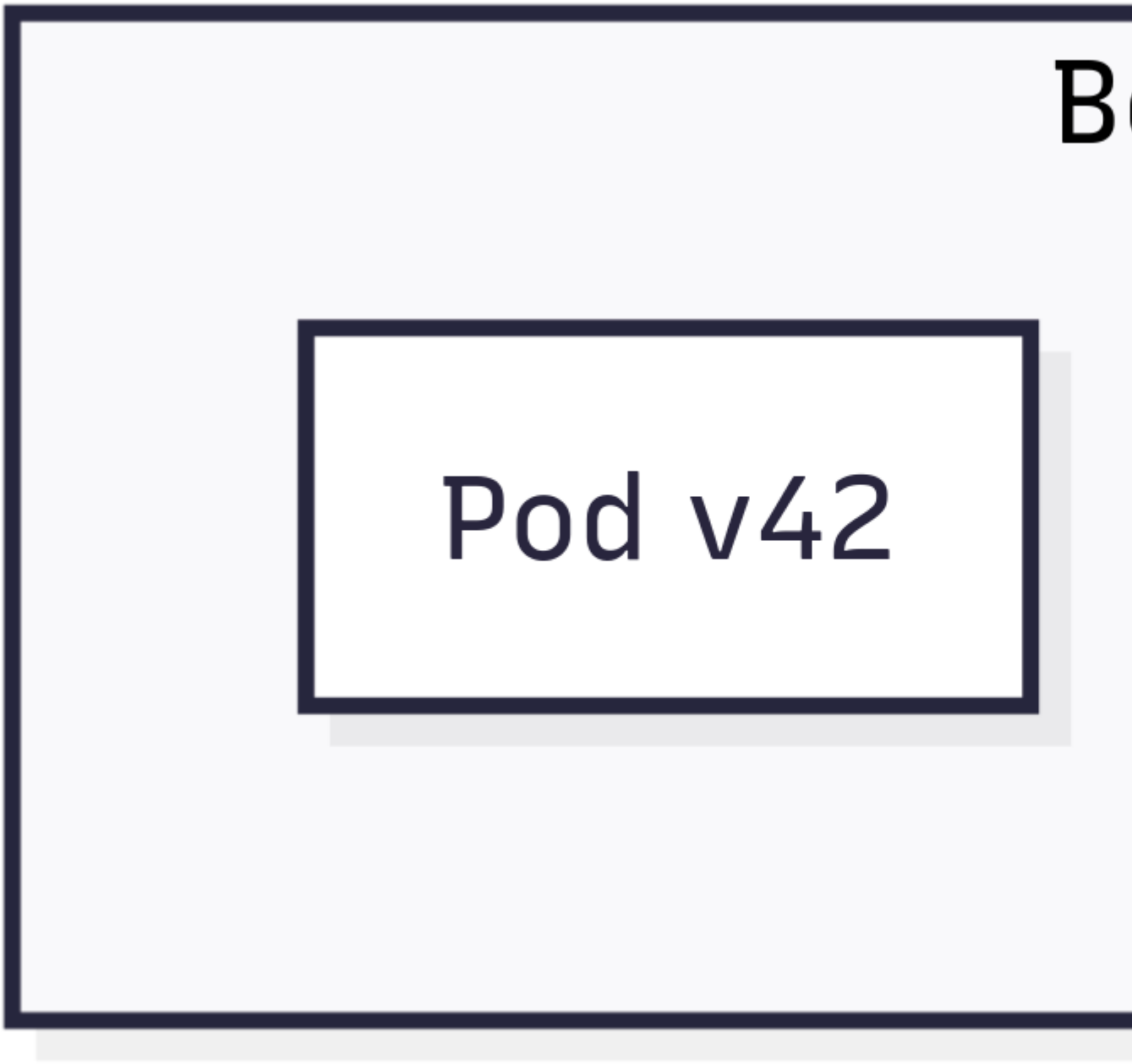
Best practice improvement: Also tag with the Git commit SHA for full traceability:

```
environment {
    GIT_SHA  = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
    IMAGE_TAG = "v${BUILD_NUMBER}-${GIT_SHA}"  // e.g. v43-a3f2c1d
}
```

Rolling Update Behaviour

When `kubectl set image` is called, Kubernetes performs a rolling update:

```
Before: [Pod v42] [Pod v42]
         ↓
Step 1: [Pod v42] [Pod v42] [Pod v43 starting...]
Step 2: [Pod v42] [Pod v43 running] ← old pod removed
Step 3: [Pod v43] [Pod v43 running]
         ↓
After:  [Pod v43] [Pod v43]
```



- Zero downtime: old pods serve traffic until new pods pass health checks
- If the new pod never becomes Ready, the rollout stalls and times out
- The `|| rollout undo` in the Jenkinsfile catches this and restores v42

This rollout behaviour is controlled by the deployment's `strategy.rollingUpdate` settings (default: `maxUnavailable: 25%`, `maxSurge: 25%`).

Troubleshooting — Problems Encountered & Solutions

This document records every significant problem hit during the initial deployment of this project. Each entry includes the exact error, the root cause, and the steps taken to fix it.

Table of Contents

---

1. Terraform `localss` typo in ALB module

Error:

```
Error: Unsupported block type
  on modules/alb/main.tf line 1:
  localss {
```

Root cause: `locals` was misspelled as `localss`. Additionally, references throughout the file used `${locals.name}` instead of the correct `${local.name}` (the block is `locals {}` but the reference keyword is `local.`).

Fix:

```
# Wrong
localss {
  name = "${var.project}-${var.environment}"
}
resource "..." {
  name = "${locals.name}-alb"
}

# Correct
locals {
  name = "${var.project}-${var.environment}"
}
resource "..." {
  name = "${local.name}-alb"
}
```

---

2. ALB module — wrong security group and subnet attribute names

Error:

```
Error: Unsupported argument
  on modules/alb/main.tf:
  "security_group_id": argument not supported here
  "subnet": argument not supported here
```

Root cause: The `aws_lb` resource uses `security_groups` (a list) and `subnets` (a list), not singular forms.

Fix:

```
# Wrong
resource "aws_lb" "main" {
  security_group_id = var.alb_sg_id
  subnet            = var.public_subnet_ids
}

# Correct
resource "aws_lb" "main" {
  security_groups = [var.alb_sg_id]
  subnets         = var.public_subnet_ids
}
```

---

3. ALB module — `redirects` block instead of `redirect`

Error:

```
Error: Unsupported block type
  on modules/alb/main.tf:
  redirects {
```

Root cause: The action block for HTTP→HTTPS redirect uses `redirect {}` (singular), not `redirects {}`.

Fix:

```
# Wrong
action {
  type = "redirect"
  redirects {
    port        = "443"
    protocol    = "HTTPS"
    status_code = "HTTP_301"
  }
}

# Correct
action {
  type = "redirect"
  redirect {
    port        = "443"
    protocol    = "HTTPS"
    status_code = "HTTP_301"
  }
}
```

---

4. ASG module — `autoscalling` typos throughout

Error:

```
Error: Invalid resource type
  on modules/asg/main.tf:
  resource "aws_autoscalling_policy" "cpu"
```

Root cause: The module was written with `autoscalling` (double-l) throughout. The correct AWS provider resource names use `autoscaling` (single-l).

Affected resources:

- `aws_autoscalling_groups` → `aws_autoscaling_groups`
- `aws_autoscalling_policy` → `aws_autoscaling_policy`
- `predefined_metric_type = "ASGAverageCPUAutoscallization"` → `"ASGAverageCPUUtilization"`
- `predefied_metric_specification` → `predefined_metric_specification`
- `locals.name` → `local.name`

Fix: Global find-and-replace of `autoscalling` → `autoscaling` across the file, plus fixing the metric name and `locals.` → `local.` references.

---

5. ASG module — `filters = [...]` syntax rejected

Error:

```
Error: Unsupported argument
  on modules/asg/main.tf:
  filters = [
```

Root cause: The `aws_autoscaling_groups` data source does not accept a `filters` list argument. It uses separate `filter {}` blocks.

Fix:

```
# Wrong
data "aws_autoscaling_groups" "eks_nodes" {
  filters = [
    { name = "tag:eks:cluster-name" values = [var.cluster_name] },
    { name = "tag:eks:nodegroup-name" values = [var.node_group_name] }
  ]
}

# Correct
data "aws_autoscaling_groups" "eks_nodes" {
  filter {
    name   = "tag:eks:cluster-name"
    values = [var.cluster_name]
  }
  filter {
    name   = "tag:eks:nodegroup-name"
    values = [var.node_group_name]
  }
}
```

---

6. Security groups — `ip_protocol` and non-list `cidr_blocks`

Error:

```
Error: Unsupported argument "ip_protocol"
Error: Incorrect attribute value type — "0.0.0.0/0" (string) cannot be used as cidr_blocks
```

Root cause: The `aws_security_group` inline `ingress`/`egress` blocks use `protocol` (not `ip_protocol`) and `cidr_blocks` must be a list, not a plain string.

`ip_protocol` is the attribute name used in the standalone `aws_vpc_security_group_ingress_rule` resource — a different resource type.

Fix:

```
# Wrong (inline ingress block)
ingress {
  ip_protocol = "-1"
  cidr_blocks = "0.0.0.0/0"
}

# Correct (inline ingress block)
ingress {
  protocol    = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
```

---

7. Security group — `aws_security_group` used instead of `aws_security_group_rule`

Error:

```
Error: Unsupported argument
  on modules/security-groups/main.tf:
  source_security_group_id is not a valid argument for aws_security_group
```

Root cause: The rule that allows ALB traffic to reach EKS nodes (`alb_to_nodes`) was mistakenly written as an `aws_security_group` resource instead of an `aws_security_group_rule` resource. Security groups cannot reference other security groups in their inline blocks — cross-SG references require a standalone `aws_security_group_rule`.

Fix:

```
# Wrong
resource "aws_security_group" "alb_to_nodes" {
  source_security_group_id = aws_security_group.alb.id
  ...
}

# Correct
resource "aws_security_group_rule" "alb_to_nodes" {
  type                     = "ingress"
  from_port                = 3000
  to_port                  = 3000
  protocol                 = "tcp"
  security_group_id        = aws_security_group.eks_nodes.id
  source_security_group_id = aws_security_group.alb.id
  description              = "ALB to EKS node port"
}
```

---

8. EKS module — dangling `tags {}` block outside any resource

Error:

```
Error: Unsupported block type
  on modules/eks/main.tf line 315:
  tags {
```

Root cause: A stray `tags {}` block appeared after the last `}` that closed the final resource. It was outside any resource and Terraform rejected it as a top-level block.

Fix: Deleted the orphaned block (lines 314–318 of the original file).

---

9. S3 lifecycle rule — missing `filter {}` block

Error:

```
Error: Missing required argument
  The argument "filter" is required for aws_s3_bucket_lifecycle_configuration rule
```

Root cause: AWS requires every S3 lifecycle rule to have a `filter {}` block specifying which objects the rule applies to. An empty `filter {}` means "apply to all objects."

Fix:

```
rule {
  id     = "abort-incomplete-multipart"
  status = "Enabled"

  filter {}   # apply to all objects

  abort_incomplete_multipart_upload {
    days_after_initiation = 7
  }
}
```

---

10. Non-ASCII em-dashes in security group descriptions rejected by AWS

Error:

```
Error: creating Security Group: InvalidParameterValue: Invalid description
  description = "EKS nodes — allow self"
```

Root cause: The `—` character (em-dash, Unicode U+2014) is not in the ASCII character set. AWS security group descriptions only accept printable ASCII characters (letters, numbers, spaces, and `_.:/-`).

Fix: Replaced all em-dashes with ASCII hyphens (`-`) in all security group description strings.

---

11. RDS — Performance Insights not supported on db.t3.micro

Error:

```
Error: modifying RDS DB Instance: InvalidParameterCombination:
  Performance Insights is not supported for DB instance class db.t3.micro
```

Root cause: AWS Performance Insights requires `db.t3.small` or larger. The dev environment uses `db.t3.micro` to minimise cost.

Fix:

```
# modules/rds/main.tf
performance_insights_enabled = false
```

Performance Insights is supported in the prod tfvars where `db_instance_class = "db.t3.small"`.

---

12. Route53 — `count` depends on unknown value during plan

Error:

```
Error: Invalid count argument
  The "count" value depends on resource attributes that cannot be determined
  until apply, so Terraform cannot predict how many instances will be created.
```

Root cause: The Route53 record was written with `count = var.alb_dns_name ≠ "" ? 1 : 0`. Because `alb_dns_name` was derived from another resource's output (not a static variable), its value was unknown at plan time — Terraform cannot evaluate the conditional.

Fix: Removed the conditional count entirely. The Route53 record is always created:

```
resource "aws_route53_record" "app" {
  # removed: count = var.alb_dns_name ≠ "" ? 1 : 0
  zone_id = data.aws_route53_zone.main.zone_id
  name    = "${var.subdomain}.${var.domain}"
  type    = "A"
  alias {
    name                   = var.alb_dns_name
    zone_id                = var.alb_zone_id
    evaluate_target_health = true
  }
}
```

13. Helm provider chicken-and-egg with EKS

Error:

```
Error: Kubernetes cluster unreachable: the server doesn't have a resource type "helmrelease"
```

or

```
Error: configuring Terraform AWS Provider: no valid credential sources found
```

Root cause: The Terraform configuration included a `provider "helm"` block that needed the EKS cluster endpoint and certificate to connect. But those values are only available after the EKS cluster is created — which happens during the same `terraform apply`. Terraform evaluates all providers before running any resources, creating a deadlock.

Fix: Removed the `provider "helm"` block and all `helm_release` resources from Terraform entirely. The Cluster Autoscaler is now installed manually via Helm CLI after the EKS cluster is up:

```
helm repo add autoscaler https://kubernetes.github.io/autoscaler
helm install cluster-autoscaler autoscaler/cluster-autoscaler \
  --namespace kube-system \
  --set autoDiscovery.clusterName=todo-tf-cluster-dev \
  --set awsRegion=us-east-1 \
  --set rbac.serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=<autoscaler-role-arn>
```

14. CloudWatch log groups already existed in AWS

Error:

```
Error: creating CloudWatch Logs Log Group: ResourceAlreadyExistsException:
  The specified log group already exists: /aws/eks/todo-tf-cluster-dev/cluster
```

Root cause: EKS automatically creates its own log group when control plane logging is enabled. When Terraform tries to create the same log group, AWS rejects it.

Fix: Import the existing log groups into Terraform state so Terraform adopts them instead of creating new ones:

```
terraform import -var-file=environments/dev/terraform.tfvars \
  module.cloudwatch.aws_cloudwatch_log_group.eks_cluster \
  /aws/eks/todo-tf-cluster-dev/cluster

terraform import -var-file=environments/dev/terraform.tfvars \
  module.cloudwatch.aws_cloudwatch_log_group.app \
  /todo/dev/application
```

After import, `terraform plan` shows no changes for these resources.

---

15. IAM roles and policies already existed in AWS

Error:

```
Error: creating IAM Role: EntityAlreadyExists: Role with name todo-dev-eks-cluster-role already exists.
Error: creating IAM Policy: EntityAlreadyExists: A policy called todo-dev-alb-controller-policy already exists
```

Root cause: A previous interrupted `terraform apply` (or manual AWS console work) had already created these IAM resources. Terraform's state file didn't know they existed, so it tried to create them again.

Fix: Import each resource into state:

```
# EKS cluster role
terraform import -var-file=environments/dev/terraform.tfvars \
  module.eks.aws_iam_role.eks_cluster todo-dev-eks-cluster-role

# EKS node role
terraform import -var-file=environments/dev/terraform.tfvars \
  module.eks.aws_iam_role.eks_nodes todo-dev-eks-node-role

# EBS CSI driver role
terraform import -var-file=environments/dev/terraform.tfvars \
  module.eks.aws_iam_role.ebs_csi todo-dev-ebs-csi-role

# ALB controller role
terraform import -var-file=environments/dev/terraform.tfvars \
  module.eks.aws_iam_role.alb_controller todo-dev-alb-controller-role

# ALB controller policy (import by ARN)
terraform import -var-file=environments/dev/terraform.tfvars \
  module.eks.aws_iam_policy.alb_controller \
  arn:aws:iam::668076964228:policy/todo-dev-alb-controller-policy
```

---

16. Route53 CNAME record conflicted with new A alias record

Error:

```
Error: [ERR]: Error building changeset: InvalidChangeBatch:
  RRSet of type CNAME with DNS name www.ankit.services. is not permitted at apex in zone ankit.services.
```

or simply the plan failing because an existing CNAME `www.ankit.services` prevented creating an A record.

Root cause: During an earlier attempt, the ALB controller had auto-created a CNAME record pointing `www.ankit.services` to the controller-managed ALB DNS name. Terraform's Route53 module was trying to create an A alias record at the same name — two records of different types at the same name conflict.

Fix: Manually deleted the old CNAME record via AWS CLI:

```
aws route53 change-resource-record-sets \
  --hosted-zone-id Z07812883CC72HQVS0WSK \
  --change-batch '{
    "Changes": [{
      "Action": "DELETE",
      "ResourceRecordSet": {
        "Name": "www.ankit.services.",
        "Type": "CNAME",
        "TTL": 300,
        "ResourceRecords": [{"Value": "k8s-default-todoingr-096e8f96c2-1251263405.us-east-1.elb.amazonaws.com"}]
      }
    }]
  }'
```

Then re-ran `terraform apply` to create the correct A alias record.

---

17. ALB controller CrashLoopBackOff — missing VPC ID and region

Symptom:

```
$ kubectl get pods -n kube-system
NAME                                           READY   STATUS             RESTARTS
aws-load-balancer-controller-xxx               0/1     CrashLoopBackOff   5

$ kubectl logs -n kube-system deployment/aws-load-balancer-controller
level=error msg="VPC ID must be specified"
```

Root cause: The Helm install command was missing required values. The ALB controller needs to know which VPC to manage and which AWS region it is running in. Without these, it crashes on startup.

Fix: Reinstall with the missing flags:

```
helm upgrade --install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=todo-tf-cluster-dev \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set vpcId=vpc-045b7a536d94660d0 \
  --set region=us-east-1
```

---

18. Webhook `context deadline exceeded` — wrong node security group

Symptom:

```
$ kubectl apply -f app/k8s/
Error from server (InternalError): error when creating "deployment.yaml":
  Internal error occurred: failed calling webhook "mpod.elbv2.k8s.aws":
  Post "https://aws-load-balancer-webhook-service.kube-system.svc:443/mutate-v1-pod":
  context deadline exceeded
```

Root cause: The Kubernetes API server routes webhook calls through the cluster's internal network. The webhook pod lives on a node with security group `sg-04f784e2a0b0c0f10` (applied by the initial failed Terraform apply). However, Terraform's security group rules were written to a new SG `sg-0910a9a61b664d82d` created by a later apply. The cluster's security group `sg-0d1c831b59c30df03` had no inbound rules allowing it to reach port 443 on the actual node SG.

Root cause in plain English: Nodes got their SG from the first (interrupted) Terraform apply. Terraform's rules went into a different SG from the second apply. The control plane couldn't reach the nodes on ports 443 (webhooks) or 10250 (kubelet).

Fix: Manually added the required rules to the actual node SG (`sg-04f784e2a0b0c0f10`) from the cluster SG (`sg-0d1c831b59c30df03`):

```
# Allow kubelet API (port 10250)
aws ec2 authorize-security-group-ingress \
  --group-id sg-04f784e2a0b0c0f10 \
  --protocol tcp --port 10250 \
  --source-group sg-0d1c831b59c30df03

# Allow webhook HTTPS (port 443)
aws ec2 authorize-security-group-ingress \
  --group-id sg-04f784e2a0b0c0f10 \
  --protocol tcp --port 443 \
  --source-group sg-0d1c831b59c30df03

# Allow high ports (1025-65535) for NodePort and ephemeral traffic
aws ec2 authorize-security-group-ingress \
  --group-id sg-04f784e2a0b0c0f10 \
  --protocol tcp --port 1025-65535 \
  --source-group sg-0d1c831b59c30df03
```

Long-term fix: Ensure Terraform manages only one node SG and does not create a second one during re-apply. Or use `terraform state rm` and re-import the correct SG so state and reality align.

---

19. ALB controller `AccessDenied: DescribeListenerAttributes`

Symptom:

```
$ kubectl logs -n kube-system deployment/aws-load-balancer-controller
AccessDenied: User: arn:aws:sts::668076964228:assumed-role/todo-dev-alb-controller-role/...
  is not authorized to perform: elasticloadbalancing:DescribeListenerAttributes
  on resource: arn:aws:elasticloadbalancing:...
```

Root cause: The IAM policy file (`alb-controller-policy.json`) was based on an older version of the AWS Load Balancer Controller. Version 3.3.0+ requires the `elasticloadbalancing:DescribeListenerAttributes` permission, which was not in the original policy.

Fix (immediate — add permission to existing policy):

```
# Get current policy ARN
POLICY_ARN=$(aws iam list-policies --query \
  "Policies[?PolicyName=='todo-dev-alb-controller-policy'].Arn" \
  --output text)

# Create a new policy version with the added permission
aws iam create-policy-version \
  --policy-arn $POLICY_ARN \
  --policy-document file://updated-policy.json \
  --set-as-default
```

Fix (permanent — update the policy file):
Added `"elasticloadbalancing:DescribeListenerAttributes"` to the `Action` list in `infra/terraform/modules/eks/alb-controller-policy.json`.

The controller recovers automatically after the IAM update — no pod restart needed.

---

20. Backend pods crashing — wrong DB_HOST and DB_PASSWORD in Secret

Symptom 1:

```
$ kubectl logs -n todo deployment/todo-backend
Error: getaddrinfo ENOTFOUND todo-db
```

Root cause 1: The `DB_HOST` value in `app/k8s/secret.yaml` was base64-encoded `todo-db` (a placeholder from earlier). The actual RDS endpoint created by Terraform is `todo-db-dev.c23qc6e80bp5.us-east-1.rds.amazonaws.com`.

Fix: Re-encode the correct hostname:

```
terraform -chdir=infra/terraform output -raw rds_endpoint | cut -d: -f1 | base64
# → dG9kby1kYi1kZXYuYzIzcWM2ZTgwYnA1LnVzLWVhc3QtMS5yZHMuYW1hem9uYXdzLmNvbQ==
```

Update `secret.yaml` with the new value.

Symptom 2:

```
$ kubectl logs -n todo deployment/todo-backend
Error: Access denied for user 'todo_user'@'...' (using password: YES)
```

Root cause 2: The initial `DB_PASSWORD` in the Secret was `rootpass` (a placeholder). The RDS master password had been set via `TF_VAR_rds_password` but the Secret was never updated to match. Additionally, an earlier attempt used `tododev` (7 characters) which AWS RDS rejected — RDS requires a minimum 8-character password.

Fix:

1. Reset the RDS master password to a valid value (minimum 8 characters):

```
aws rds modify-db-instance \
  --db-instance-identifier todo-db-dev \
  --master-user-password tododev1 \
  --apply-immediately
```

1. Wait ~60 seconds for the password change to propagate.
2. Re-encode the new password and update the Secret:

```
echo -n "tododev1" | base64
# → dG9kb2RldjE=
```

1. Apply the updated Secret:

```
kubectl apply -f app/k8s/secret.yaml
kubectl rollout restart deployment/todo-backend -n todo
```

Key Lessons

#	Lesson
1–4	Small typos in Terraform (double-l, wrong prefix) cause cryptic errors — validate with `terraform validate` before apply
5–7	Check the AWS provider documentation for exact attribute names — the Terraform registry docs are authoritative
8	Every `{}` block must be inside a resource; stray top-level blocks break the whole file
10	AWS APIs only accept printable ASCII — copy-pasting from word processors introduces invisible characters
11	Instance class constraints exist for many AWS features — check compatibility before enabling
12–13	Terraform `count` and provider blocks cannot depend on values unknown at plan time
14–16	When re-running Terraform on an account that already has resources, `terraform import` is the right tool — never delete-and-recreate manually managed resources
17	Read the controller's startup logs before assuming the issue is network or IAM — the error message is usually specific
18	Security groups must match the actual SG assigned to nodes, not the SG Terraform thinks it assigned
19	Keep IAM policies up to date with the controller version — new permissions are added in minor releases
20	Always encode real values in Secrets before deploying — placeholder base64 strings will silently connect to nonexistent hosts